

UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

DIPARTIMENTO DI SCIENZE TEORICHE E APPLICATE

CORSO DI STUDIO TRIENNALE IN
INFORMATICA

Migrazione e sviluppo di un'applicazione Android per l'analisi dei melanoma

Relatore:
Prof. Ignazio Gallo

Tesi di Laurea di:
Gabriele Caretti
Matricola 756564

Anno accademico:
2025/2026

Indice

1 Introduzione **1**
1.1 Obiettivo 1
1.2 Melanoma 1

2 Analisi dei Requisiti **3**

3 Progettazione **5**
3.1 Descrizione generale 5
3.2 Scelte architetturali 5
3.2.1 Migrazione verso Kotlin Multiplatform 5
3.2.2 Architettura dell'applicazione 6
3.3 Specifica dei requisiti 7
3.3.1 Interfaccia utente 7
3.3.2 Realizzazione del layout 8
3.4 Database 9
3.4.1 Modello dati 10
3.4.2 Struttura record diagnosi 11
3.4.3 Interazione con Firestore 11

4 Sviluppo **13**
4.1 Implementazione interfaccia con Compose Multiplatform 13
4.2 Creazione base dei dati 14
4.3 Sicurezza 14

5 Conclusioni **15**

Capitolo 1

Introduzione

1.1 Obiettivo

Il progetto di tesi si propone di sviluppare un'applicazione mobile cross-platform, compatibile sia su piattaforme Android che iOS, partendo dallo sviluppo di un'applicazione nativa scritta in Android Kotlin, che necessità di una migrazione verso Kotlin Multiplatform. L'applicazione nativa offre analisi tramite l'input di parametri dei tumori di tipo Melanoma, con metrica Melanoma TNM, seguendo il manuale AJCC 8th Edition. Questa sarà successivamente estesa con l'implementazione di analisi di tipo cSCC Testa-Collo, con metrica cSCC TNM, sempre seguendo il precedente manuale AJCC 8th Edition.

L'obiettivo è quello di offrire uno strumento semplice e veloce per semplificare il lavoro dei medici nella fase di analisi dei tumori, con dati sincronizzati in ambiente cloud tramite app installabili su dispositivi mobili.

Il processo di realizzazione del software è strutturata in una sequenza lineare di fasi, nello specifico:

- Analisi dei requisiti;
- Progettazione;
- Sviluppo;
- Conclusioni;

1.2 Melanoma

Il melanoma è la forma più aggressiva e pericolosa di tumore cutaneo. Si origina dai *melanociti*, cellule presenti sull'epidermide responsabili della produzione di

melanina, ossia il pigmento che ne da il colore e la protegge dai raggi solari, come ultravioletti (UVA, UVB e UVC). Quando il DNA di queste cellule si danneggia (spesso a causa di un'eccessiva esposizione ai componenti pericolosi dei raggi solari), i melanociti possono iniziare a moltiplicarsi in modo incontrollato, formando quindi una massa tumorale.

A livello visivo, un melanoma può manifestarsi come la comparsa di un "neo" anomalo oppure come il cambiamento di forma, colore o dimensione di uno già esisteste. Sebbene i melanomi rappresentino solamente una piccola percentuale di tutti i tumori cutanei, questi sono responsabili della maggioranza dei decessi legati a tal genere di patologie. Questo è spesso dovuto alla loro tendenza di diffondersi nei tessuti profondi e a generare metastasi in altri organi se non individuati in tempo.

Una diagnosi precoce e precisa può quindi salvare la vita di un individuo affetto da melanoma. In ambito clinico, questo viene valutato attraverso il **sistema di stadiazione TNM** (Tumore, LinfoNodi, Metastasi), un metodo standardizzato ed esposto in manuali come l'AJCC 8th Edition menzionato in precedenza, che ne definiscono con importanza critica la prognosi e il piano terapeutico del paziente.

In un panorama ampio come quello dell'oncologia dermatologica, un'altra importante manifestazione di tale patologia è il **Carcinoma a Cellule Squamose (SCC)**. Quest'ultimo, specialmente nei casi in cui è localizzato nel distretto testa-collo, richiede un monitoraggio costante e attento, a causa del suo potenziale impatto distruttivo a livello funzionale ed estetico.

Questo progetto di applicazione mobile, mira quindi allo sviluppo di uno strumento di supporto digitale, che ne semplifica le fasi di diagnosi, con classificazione, stadiazione e storico dei pazienti affetti da melanoma TNM e cSCC TNM Testa-Collo.

Capitolo 2

Analisi dei Requisiti

Una delle prime attività fondamentali dell'ingegneria del software per la buon riuscita di un prodotto, è l'analisi dei requisiti. Questa fase ha lo scopo di individuare e definire le funzionalità offerte che saranno sviluppate.

La nostra applicazione mobile dovrà quindi offrire:

- Fase di apertura, con caricamento dell'interfaccia principale per l'inserimento dei parametri
- Tema dinamico, tramite pulsante per il passaggio da tema chiaro a scuro e viceversa, anche in base alle impostazioni del dispositivo
- Multilingua, inglese e italiano, anch'esse in base alle impostazioni del dispositivo
- Risultati con spiegazione delle analisi, con eventuale follow-up consigliato
- File di configurazione con cui definire parametricamente i risultati che gli input dovranno dare
- Funzionalità di salvataggio dei risultati di diagnosi dei pazienti, con possibilità di visualizzarli in uno storico con ogni loro diagnosi
- Funzionalità di cancellazione dei paziente, inclusi il loro storico, o delle loro singole diagnosi
- Funzionalità per passare da una diagnosi TNM a sSCC Testa-Collo e viceversa
- Funzionalità di caricamento delle diagnosi precedenti, senza dover reinserire i parametri manualmente

- Sincronizzazione con un servizio cloud, per salvare i dati dei pazienti e le loro diagnosi

L'app mobile deve inoltre soddisfare dei requisiti non funzionali, come:

- UI/UX semplice, intuitiva e moderna, senza complicare più del necessario l'utilizzo dell'utente finale
- Sicurezza, sia locale che cloud
- Funzionalità offline, in assenza di internet l'app deve comunque permettere di eseguire diagnosi

Capitolo 3

Progettazione

Successivamente alla fase di analisi dei requisiti, la fase di progettazione definisce come tali requisiti saranno effettivamente implementati, sia a livello di struttura software che di interfaccia utente.

3.1 Descrizione generale

3.2 Scelte architetturali

3.2.1 Migrazione verso Kotlin Multiplatform

Il progetto è stato inizialmente sviluppato in Kotlin nativo Android, utilizzando strumenti Java e non solo, come:

- EvalEx, per la lettura dei file delle espressioni per ottenere le diagnosi
- Firebase, per la sincronizzazione dei dati dei pazienti e delle diagnosi
- SpashScreen, per l'animazione iniziale di caricamento
- Lingua e tema di default in base al dispositivo

Questi elementi sono stati sostituiti con altre librerie simili o sostituiti, nello specifico:

- EvalEx è stato rimosso, e si è invece scritto un interprete da zero nativo Kotlin
- Firebase è stato sostituito con la version Gitlive della community, con l'aggiunta di diverse dipendenze per i servizi Google

- Lo SplashScreen animato è stato rimosso in favore di uno statico
- La lingua e il tema di default sono stati scritti nelle rispettive iosMain e androidMain, in quanto diverse per piattaforma

3.2.2 Architettura dell'applicazione

L'architettura del software si adatta agli standard di Kotlin Multiplatform (KMP). Durante la migrazione da Kotlin a Kotlin Multiplatform, l'obiettivo primario è stato quello di massimizzare il codice condiviso tra Android e iOS, con il minor numero di dipendenze native ove possibile, con una logica di business e di presentazione (UI) unificati.

L'applicazione segue una variante del pattern architetturale **Model-View-ViewModel (MVVM)**, costruita attorno a una singola *ViewModel* centralizzata (`MelanomaViewModel`). Questa funge da singolo punto di accesso per l'intero ciclo di vita dell'applicazione, separando l'interfaccia utente dalla logica di dominio.

Le caratteristiche architetturali principali includono:

- Utilizzo di Compose Multiplaform, compatibile sia con Android che iOS in Kotlin Multiplatform, tramite il modulo condiviso `commonMain`. Le viste (View) sono *stateless*, e si limitano quindi a osservare i dati e a inoltrare al ViewModel le interazioni dell'utente.
- Utilizzo di `StateFlow` e di `MutableStateFlow` per la gestione della reattività degli stati. Variabili come dati anagrafici, parametri clinici e risultati della stadiazione (`StagingResult`) vengono aggiornati reattivamente. Viene quindi ricomposta automaticamente l'interfaccia grafica quando i flussi di dati cambiano, emettendo un nuovo stato.
- Navigazione tra le schermate gestita tramite stato (State-Driven), senza librerie di routing esterne (inizialmente nella versione nativa veniva utilizzato il Navigation Component di Android), viene quindi gestita in modo manuale e dichiarativo la navigazione tra le schermate. Una `Sealed class Screen` definisce le possibili destinazioni navigabili dell'app (`Calculator`, `PatientList` e `PatientHistory`). Il ViewModel mantiene lo stato della schermata attiva, e fornisce la funzione `navigateTo(screen: Screen)`.

Cambiando lo stato della variabile, viene quindi cambiata la schermata, eseguendo eventuale logica annessa (come il caricamento dei pazienti tramite fetch).

- Utilizzo di `Kotlin Coroutines` per la gestione della concorrenza e dell'accesso ai dati. Tutte le operazioni asincrone, come il calcolo della diagnosi basato su regole dei file JSON (e tramite `StagingEngine`), inclusa la comunicazione cloud di Firebase, sono gestite in modo sicuro e non bloccante, anche tramite il `viewModelScope`. L'interazione con il database cloud Firebase, inizialmente con Firebase nativo di Google, è stata migrata alla versione KMP della community GitLive, che permette di utilizzare funzioni CRUD (lettura, salvataggio e cancellazione) sui dati senza dover applicare codici diversi in base al sistema operativo.

In breve, la maggior parte del codice si trova in commonMain e utilizza codice Kotlin compatibile sia con sistemi Android che iOS, con il minimo utilizzo di funzioni proprietare tramite `expect/actual`, lasciando quindi i moduli `androidMain` e `iosMain` il più snelli possibili (salvo inizializzazioni e configurazioni iniziali obbligatorie).

3.3 Specifica dei requisiti

3.3.1 Interfaccia utente

L'interfaccia utente (UI) e l'esperienza utente (UX) sono state progettate per essere intuitive e di rapido utilizzo a livello clinico, essendo questa applicazione uno strumento di supporto per i medici, deve essere semplice e con un carico cognitivo minimo per l'inserimento dei dati e la lettura dei risultati delle diagnosi.

Per definire gli Screen (schermate) e definire un progetto dell'applicazione prima della scrittura del codice, è stato utilizzato **Figma**. Il prototipo è stato costruito secondo i seguenti criteri:

- Parametri suddivisi logicamente in sezioni ben definite e distinte (Parametri T, N e M), racchiuse in `Card` con stili personalizzati.

- Pulsanti di input standard, come *switch*, *chip* e *checkbox* selezionabili, al posto di opzioni più complesse come menù a tendina. Alcuni switch o checkbox quando premuti mostrano ulteriori parametri opzionali od obbligatori, per l'inserimento di dati specifici per le diagnosi.
- Supporto sia al tema chiaro che scuro, per non affaticare la vista dell'utente in ambito ospedaliero con diverse condizioni di luce. Viene inoltre provvisto direttamente nella barra di navigazione un pulsante per il passaggio veloce tra i temi.

3.3.2 Realizzazione del layout

Inizialmente il layout è stato realizzato seguendo un prototipo senza cSCC Testa-Collo, ma solamente con la stadiazione del melanoma TNM. Successivamente ha subito un'estensione per supportare il nuovo tipo di diagnosi e introdurre la compatibile con Kotlin Multiplatform. Questo è visibile anche nei prototipi Figma (Figura 3.1 e Figura 3.2):

In (Figura 3.1), sviluppata in Kotlin Android Nativo, il layout e l'interfaccia erano strutturati unicamente per supportare la stadiazione del Melanoma TNM, con parametri statici e senza selettori per altri tipi di diagnosi.

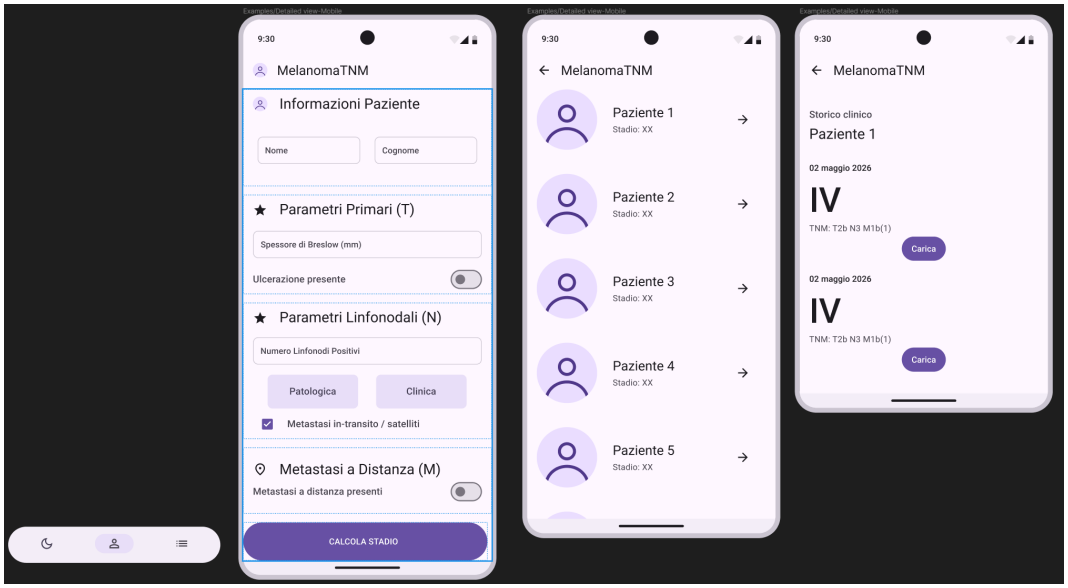


Figura 3.1: Fase 1 (Android Nativo) - Solo Melanoma

In (Figura 3.2), con la migrazione verso Kotlin Multiplatform, si è aggiunta anche l'opzione di diagnosi del Carcinoma a Cellule Squamose (cSCC) Testa-Collo, di conseguenza è stata aggiunta una nuova `card`, con due pulsanti di

selezione per scegliere il tipo di diagnosi (*Tipo di tumore*). Quando premuti, ricaricano dinamicamente la sezione dei parametri con i rispettivi tipi T, N e M.

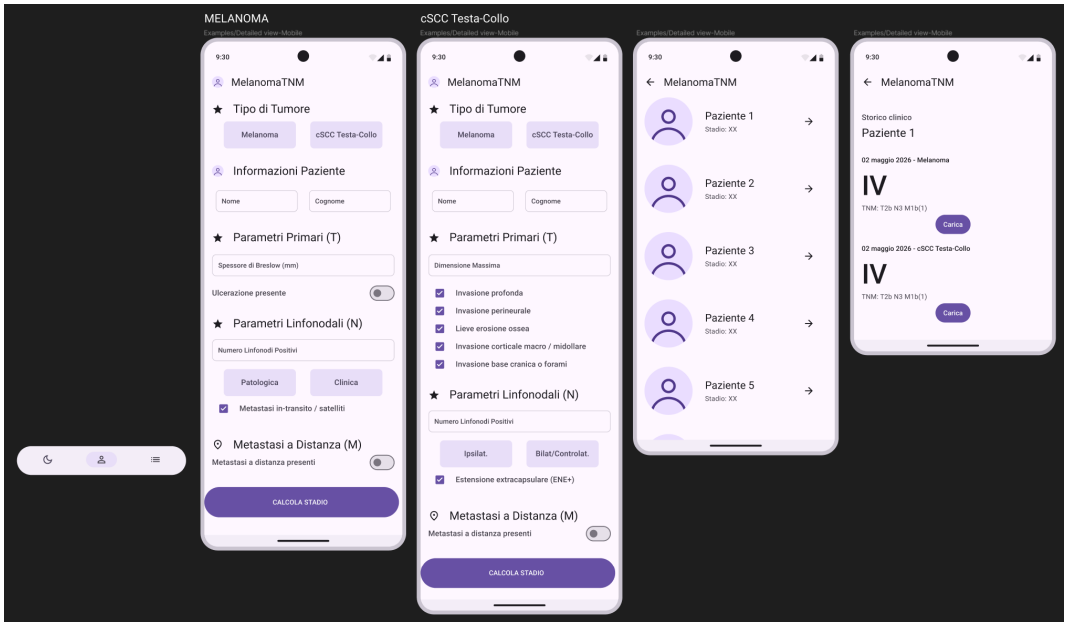


Figura 3.2: Fase 2 (KMP) - Selezione Dinamica

Il progetto definito in Figma è stato successivamente tradotto in codice con **Compose Multiplatform** (nella prima versione invece con **Jetpack Compose**). Sono stati creati e utilizzati dei componenti personalizzati come `ModernChip` (un selettore) e componenti di animazione come `AnimatedContent` per ottenere delle transizioni fluide senza interruzioni. Grazie a questi componenti personalizzati si è resa più modulare l'applicazione e quindi anche il suo mantenimento per eventuali aggiornamenti futuri.

3.4 Database

Utilizziamo **Cloud Firebase Firestore** per la persistenza e sincronizzazione dei dati nel cloud. Firestore è un database NoSQL a documenti offerto nell'ecosistema Firestore di Google, tuttavia dato che questo non è direttamente supportato su Kotlin Multiplatform, come menzionato in precedenza durante la fase di migrazione, è stato necessario adottare la libreria SDK *Gitlive Firebase* della community, che offre funzionalità simili e compatibilità con Firestore e KMP. Essa offre dei *wrapper* Kotlin sia per Android che iOS, che possono essere utilizzati direttamente nel modulo `commonMain`.

Essendo un database NoSQL a documenti, esso ha un funzionamento diverso dai tradizionali database relazionali con tabelle, righe e colonne. Firestore mantiene i dati in una struttura gerarchica di *documenti* in formato simile a JSON, raggruppati in *collezioni*, che a loro volta contengono altri documenti, con le proprie raccolte.

3.4.1 Modello dati

Utilizziamo una struttura progettata attorno a un'unica collezione root che abbiamo denominato `patients`. I documenti al suo interno rappresentano i singoli pazienti, identificati in modo univoco tramite UUID (*Universally Unique Identifier*). Ogni paziente ha un documento che contiene campi come lo storico (un array di diagnosi) e dati anagrafici di base.

I documenti NoSQL sono generati direttamente da codice convertendo delle `data class` Kotlin in un formato compatibile con Firestore, utilizzando la libreria `kotlinx.serialization`, che ne permette la serializzazione e deserializzazione. Di fatto, grazie all'annotazione `@Serializable`, il framework si occupa automaticamente della traduzione dei dati in documenti Firestore, fortemente tipizzati garantendo la sicurezza dei tipi dei dati e riducendo codice *boilerplate*.

Il modello dei dati principale della classe `Patients` è la seguente:

- `id`, `name`, `surname`: Stringhe che compongo i campi per l'anagrafica e l'identificazione di ogni paziente
- `birthDate`: Un campo opzionale, che rappresenta la data di nascita del paziente, e che viene gestita tramite la libreria `kotlinx.datetime`
- `history`: Un array, simile a una lista di oggetti, di `StagingRecord`

Nonostante sarebbe stato possibile modellare lo storico dei pazienti in una sotto-collezione separata, dato che stiamo lavorando in NoSQL si è deciso di mantenerlo come un array `embedded` all'interno dei documenti dei pazienti. Questo permette di ottimizzare prestazioni e costi che potrebbero derivare da multiple query su diverse collezioni. Il caricamento dei pazienti e dei relativi storici, avviene automaticamente tramite la funzione `fetchPatients()` nel

ViewModel, insieme al proprio storico clinico, in una singola operazione di lettura (read). Questa funzione viene solitamente richiamata al momento dell'apertura della schermata dei pazienti, durante la navigazione tramite bottom bar, in una `Kotlin Coroutine`.

3.4.2 Struttura record diagnosi

Gli elementi dell'array `history`, che compongono essenzialmente un'istanza della classe `StagingRecord` serializzata, sono composti da:

1. **Metadati**: Un `UUID` univoco insieme a un `timestamp` (di tipo `Instant`) che identificano e registrano il momento esatto in cui l'analisi è stata effettuata

2. **`input` MelanomaInput**: Una classe che contiene i parametri clinici inseriti dall'utente, che supporta sia la stadiazione del melanoma TNM che cSCC. I dati includono tipi primitivi come `breslowMm` e `linfonodiPositivi`, serializzati come stringhe sul database.

3. **`result` StagingResult**: Il risultato di output che viene generato dal motore di calcolo (`StagingEngine`), che memorizza i valori TNM calcolati, il follow-up suggerito e lo stadio complessivo. Infatti, al posto di salvare stringhe statiche per le spiegazioni, salviamo una chiave (come `explanationKeysM`), che permette di assegnare testi diversi in più lingue, senza salvare nel database stringhe statiche intraducibili o tradotte in più lingue, sarà poi l'interfaccia utente direttamente dal dispositivo a tradurre i valori chiave nel testo equivalente.

3.4.3 Interazione con Firestore

Il collegamento e interazione con il database vengono gestire dal `MelanomaViewModel` tramite delle *Kotlin Coroutines* permettendo di eseguire operazione `IO` senza bloccare il thread principale dell'interfaccia grafica in modo asincrono.

Gitlive Firestore, inoltre, ha delle espressioni CRUD (Create, Read, Update e Delete) molto semplici. Operazioni come il salvataggio e l'aggiornamento dei pazienti sono convertite tramite oggetti Kotlin, per esempio:

```
db.collection("patients").document(patientId).set(newPatient)
```

Quando viene necessario modificare solamente parzialmente dei dati (come eliminare una singola diagnosi dello storico), viene utilizzata l'estensione `updateFields` ottimizzata da Firestore, che permette di eseguire una modifica esclusivamente su un elemento dell'array richiesto (nel nostro caso l'array `history`) senza rieseguire una sovrascrittura dell'intero documento, un approccio che migliora l'efficienza nella gestione dei dati in ambiente cloud.

Capitolo 4

Sviluppo

4.1 Implementazione interfaccia con Compose Multiplatform

L'implementazione dell'interfaccia definita e progettata nella fase di design (Capitolo 3) è stata effettuata tramite *Compose Multiplatform*. Il framework ha una natura dichiarativa, e si è quindi sfruttato un `ViewModel` contenente gli stati che i componenti osservano e si aggiornano (ricompongono) quando cambiano.

La gestione dinamica, per esempio, tra l'ìnput del Melanoma TNM o cSCC Testa-Collo, viene ottenuta vincolando un componente (`AnimatedContent`) allo stato della variabile `tipoTumore`, avviando una transizione e animazione tra i due.

```
AnimatedContent(
    targetState = tipoTumore,
    transitionSpec = {
        (fadeIn(animationSpec = tween(220, delayMillis = 90)) +
         scaleIn(initialScale = 0.92f))
        .togetherWith(fadeOut(animationSpec = tween(90)))
    },
    label = "TumorTypeT"
) { targetType ->
    if (targetType == TipoTumore.MELANOMA) {
        // Mostra input Melanoma
        ModernTextField(
            value = breslow,
            onValueChange = { viewModel.breslowInput.value = it },
```

```
            label = stringResource(Res.string.label_breslow)
        )
    } else {
        // Mostra input cSCC
        ModernTextField(
            value = dimensioneScc,
            onValueChange = { viewModel.dimensioneSccInput.value = it },
            label = stringResource(Res.string.label_max_dimension)
        )
    }
}
```

Si è resa più fluida l'interfaccia anche grazie all'utilizzo di animazioni di transizione come `fadeIn`, `scaleIn` e `fadeOut`. Inoltre, nascondendo i componenti inutilizzati si ottimizza la UI istanziando solamente ciò che è realmente visibile a schermo, migliorando anche performance ed esperienza utente.

4.2 Creazione base dei dati

4.3 Sicurezza

Capitolo 5

Conclusioni